

Lab 5: Value-Based Reinforcement Learning

- Student ID: 1811232011(TAICA)
- Name: 林承漢ntroduction

This experiment implements and investigates value-based reinforcement learning algorithms, specifically Deep Q-Network (DQN) and its variants. DQN combines Q-learning with deep neural networks, enabling the handling of high-dimensional observation spaces in reinforcement learning problems. The experiment is divided into three main parts:

1. **Task 1:** Implementing basic DQN in the CartPole-v1 environment, a classic control problem with a low-dimensional state space.
2. **Task 2:** Extending DQN to the Atari Pong-v5 environment, dealing with high-dimensional visual inputs.
3. **Task 3:** Integrating advanced DQN enhancement techniques (Double DQN, Prioritized Experience Replay, Multi-Step Return) to improve sample efficiency.

In this experiment, we explored the performance and learning efficiency of these algorithms in different environments and compared the effectiveness of various enhancement techniques. The ultimate goal was to understand how to improve DQN's sample efficiency and convergence performance.

Implementation

Bellman Error Calculation in DQN

The Bellman error is a core concept in DQN training, measuring the gap between current Q-value estimates and target Q-values. In my code, the Bellman error calculation is as follows:

```
# Calculate target Q-values  
with torch.no_grad():
```

```

next_q_values = self.target_net(next_states).max(1)[0]
target_q_values = rewards + (1 - dones) * self.gamma * next_q_values

# Calculate current Q-values
q_values = self.q_net(states).gather(1, actions.unsqueeze(1)).squeeze(1)

# Bellman error
td_errors = target_q_values - q_values

```

The key here is using the target network to calculate the Q-values of the next state, which helps stabilize the training process. The Bellman error measures the difference between the current Q-value and the "immediate reward + discounted maximum Q-value of the next state."

Modification from DQN to Double DQN

Double DQN reduces the overestimation of Q-values by decoupling action selection and Q-value evaluation. My implementation is as follows:

```

with torch.no_grad():
    # Online network selects action
    next_actions = self.q_net(next_states).argmax(1)
    # Target network evaluates action value
    next_q_target = self.target_net(next_states).gather(1, next_actions.unsqueeze(1)).squeeze(1)
    target_q_values = rewards + (1 - dones) * self.gamma_n * next_q_target

```

The difference from the original DQN is:

1. Using the online network (Q-Network) to select the optimal action
2. Using the target network (Target Network) to evaluate the Q-value of that action
3. Separating the "selection" and "evaluation" steps, reducing overoptimistic estimates caused by network error accumulation

Implementation of Prioritized Experience Replay

Prioritized Experience Replay (PER) samples based on the importance of samples rather than uniformly at random, improving learning efficiency. My PER implementation includes:

```
class PrioritizedReplayBuffer:
    def __init__(self, capacity, alpha=0.6, beta=0.4):
        self.capacity = capacity
        self.alpha = alpha# Parameter controlling priority usage
        self.beta = beta# Importance sampling weight parameter
        self.buffer = []
        self.priorities = np.zeros((capacity,), dtype=np.float32)
        self.pos = 0
        self.beta_start = beta

    def add(self, transition, error=None):
        # Set priority for new sample
        if error is None:
            priority = self.priorities.max() if len(self.buffer) else 1.0
        else:
            priority = abs(error) + 1e-6

        # Add sample to buffer
        if len(self.buffer) < self.capacity:
            self.buffer.append(transition)
        else:
            self.buffer[self.pos] = transition
            self.priorities[self.pos] = priority
            self.pos = (self.pos + 1) % self.capacity

    def sample(self, batch_size):
        # Calculate sampling probability based on priority
        if len(self.buffer) == self.capacity:
            prios = self.priorities
        else:
            prios = self.priorities[:len(self.buffer)]
```

```

scaled_prios = prios ** self.alpha
probs = scaled_prios / scaled_prios.sum()

# Sample indices based on probability
indices = np.random.choice(len(self.buffer), batch_size, p=probs)
transitions = [self.buffer[idx] for idx in indices]

# Calculate importance sampling weights
weights = (len(self.buffer) * probs[indices]) ** (-self.beta)
weights /= weights.max()# Normalize weights
weights = torch.tensor(weights, dtype=torch.float32)

# Return samples and related information
states, actions, rewards, next_states, dones = zip(*transitions)
return (np.array(states), np.array(actions), np.array(rewards, dtype=np.float32),
        np.array(next_states), np.array(dones, dtype=np.float32),
        indices, weights)

def update_priorities(self, indices, errors):
# Update priorities based on new TD errors
self.priorities[indices] = np.abs(errors) + 1e-6

```

The implementation of PER includes three key parts:

1. **Priority assignment:** Based on the absolute value of TD error
2. **Priority-based sampling:** Using the α parameter to control the degree of priority influence
3. **Importance sampling weights:** Using the β parameter to correct sampling bias

In PER, the β value gradually increases with training to progressively reduce bias correction:

```

self.memory.beta = min(1.0, self.memory.beta_start +
                        (1.0 - self.memory.beta_start) * self.env_count / 200000)

```

Modification from 1-step to Multi-step Return

Multi-step return improves return estimation by considering rewards from multiple future time steps. My implementation uses n-step reward accumulation and target network estimation:

```
# Initialize n-step queue
self.n_step = args.n_step
self.gamma_n = self.gamma ** self.n_step
self.nstep_queue = deque(maxlen=self.n_step)

# Collect n-step transitions during environment interaction
self.nstep_queue.append((state, action, reward))
if len(self.nstep_queue) == self.n_step:
    R_n = sum((self.gamma ** i) * self.nstep_queue[i][2] for i in range(self.n_step))
    state_0, action_0, _ = self.nstep_queue[0]
    self.memory.add((state_0, action_0, R_n, next_state, done))
```

The advantages of this approach are:

1. Reducing bias in value estimation, especially in environments with sparse rewards
2. Enabling reward signals to propagate forward more quickly
3. Balancing the bias-variance trade-off, making it more efficient than 1-step TD learning

For handling terminal states, I added additional processing for remaining steps:

```
if done:
    while self.nstep_queue:
        R_n = sum((self.gamma ** i) * self.nstep_queue[i][2] for i in range(len(self.nstep_queue)))
        state_0, action_0, _ = self.nstep_queue[0]
        self.memory.add((state_0, action_0, R_n, next_state, done))
        self.nstep_queue.popleft()
```

Using Weights & Biases to Track Model Performance

To effectively track and visualize the training process, I extended Weights & Biases (wandb). Key metrics include:

```
# Logging during environment interaction
wandb.log({
    "env_step": self.env_count,
    "progress/episode": self.episode,
    "progress/step_count": step_count,
    "progress/env_step_count": self.env_count,
    "progress/update_count": self.train_count,
    "agent/epsilon": self.epsilon,
    "agent/buffer_size": len(self.memory)
})

# Logging at episode end
wandb.log({
    "env_step": self.env_count,
    "progress/episode": self.episode,
    "performance/episode_reward": total_reward,
    "progress/env_step_count": self.env_count,
    "progress/update_count": self.train_count,
    "agent/epsilon": self.epsilon,
    "performance/episode_length": step_count
})

# Logging during training
wandb.log({
    "train_step": self.train_count,
    "train/loss": loss.item(),
    "train/q_mean": q_values.mean().item(),
    "train/q_std": q_values.std().item()
})

# Logging during evaluation
```

```
wandb.log({
    "env_step": self.env_count,
    "progress/env_step_count": self.env_count,
    "progress/update_count": self.train_count,
    "evaluation/episode_reward": eval_reward,
    "evaluation/best_reward": self.best_reward,
    "Episode Reward vs Env Steps": eval_reward# Direct mapping to the chart name required by the task
})
```

I also defined metric groupings and step benchmarks to make wandb charts more organized:

```
wandb.define_metric("env_step")
wandb.define_metric("train_step")
wandb.define_metric("progress/*", step_metric="env_step")
wandb.define_metric("performance/*", step_metric="env_step")
wandb.define_metric("evaluation/*", step_metric="env_step")
wandb.define_metric("train/*", step_metric="train_step")
```

Additionally, I implemented checkpoint saving and loading mechanisms, as well as configuration tracking, to ensure experiment reproducibility and continuity.

Analysis and Discussions

Task 1: Vanilla DQN in CartPole Environment

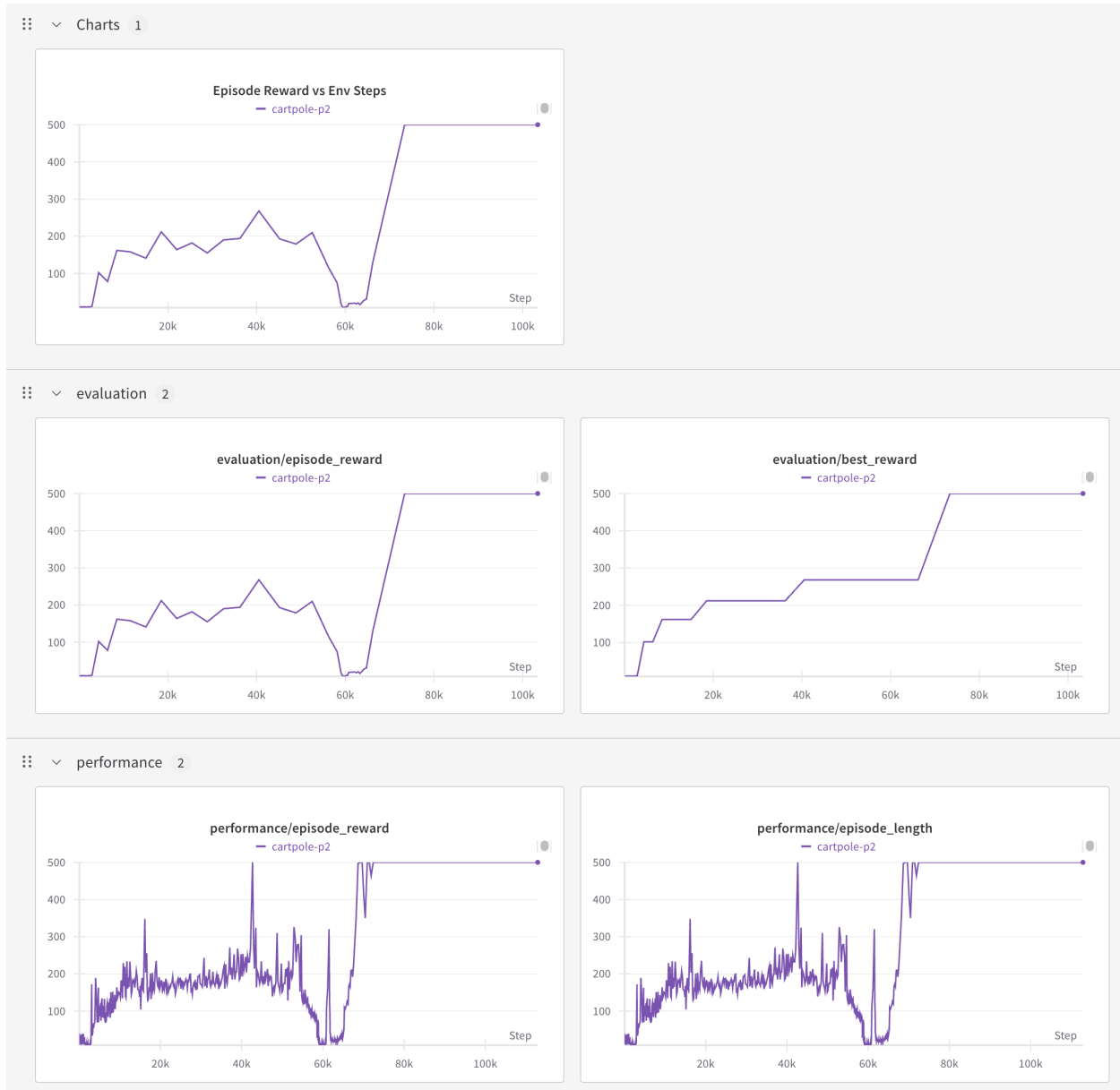
In the CartPole-v1 environment, my Vanilla DQN implementation used a simple two-layer fully connected network:

```
self.features = nn.Sequential(
    nn.Linear(input_shape[0], 128), nn.ReLU(),
    nn.Linear(128, 128), nn.ReLU()
)
```

The most critical hyperparameter settings during training included:

- Learning rate: 0.0001
- Batch size: 128
- Discount factor: 0.99
- Epsilon starting value: 1.0, linearly decaying to 0.05
- Target network update frequency: 1000 steps

The goal of the CartPole task is to keep the pole upright on the cart without falling. My DQN implementation stabilized at a reward of 500 (maximum score) after 73K steps.



Task 2: Vanilla DQN in Pong Environment

In the Pong environment, I used a convolutional neural network to process visual input:

```
python
self.features = nn.Sequential(
    nn.Conv2d(input_shape[0], 32, 8, 4), nn.ReLU(),
```

```
nn.Conv2d(32, 64, 4, 2), nn.ReLU(),  
nn.Conv2d(64, 64, 3, 1), nn.ReLU(),  
nn.Flatten(),  
nn.Linear(64*7*7, 512), nn.ReLU(),  
nn.Linear(512, num_actions)  
)
```

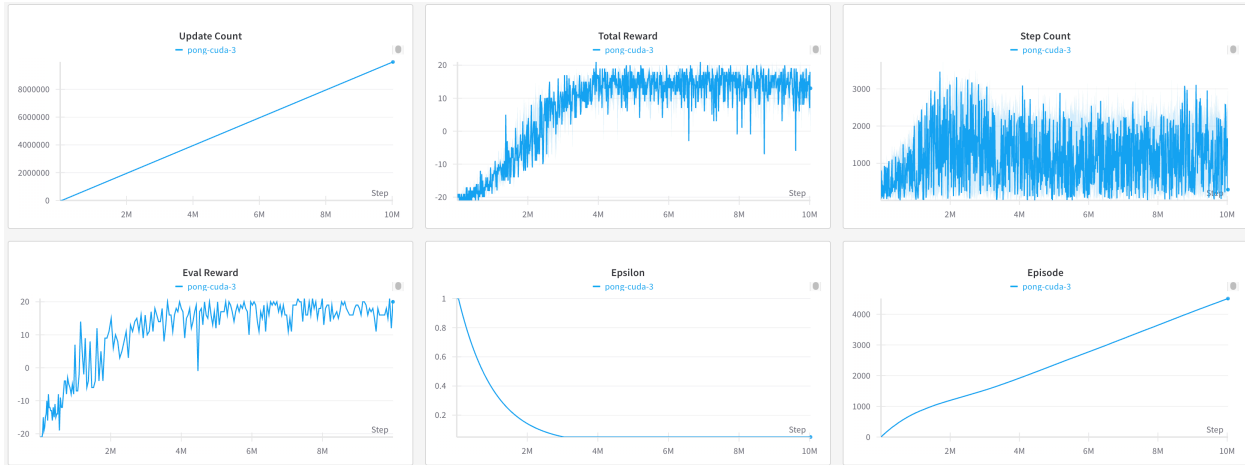
Key preprocessing steps included:

- Converting RGB images to grayscale
- Cropping and resizing to 84×84
- Stacking 4 frames as network input

Hyperparameter settings:

- Learning rate: 0.0001
- Batch size: 32
- Discount factor: 0.99
- Memory size: 250000
- Epsilon linear decay: from 1.0 to 0.05 over 1M steps

The Pong environment is more challenging than CartPole and requires longer training time. From the chart, we can see that the training initially yielded negative scores (around -21), gradually improving as training progressed. After about 2M environment steps, the model began to achieve positive scores; at 3-4M steps, performance reached 10+ points; and finally, after 4M steps, it stabilized at a high score of 15-20 points. The evaluation reward curve shows that the model could consistently achieve scores close to the maximum (21 points). A total of approximately 4500 episodes were trained, with epsilon reaching its minimum value of 0.05 at 4M steps. This result indicates that, despite slow initial learning, Vanilla DQN can learn near-optimal strategies in the Pong environment with sufficient training.



Task 3: Enhanced DQN

In Task 3, I integrated three enhancement techniques: Double DQN, Prioritized Experience Replay (PER), and Multi-step Return. Additionally, I implemented a Dueling DQN architecture to improve representational capacity by separating state value and advantage functions:

```
python
# Dueling architecture
self.value = nn.Sequential(
    nn.Linear(feature_size, 256), nn.ReLU(),
    nn.Linear(256, 1)
)
self.advantage = nn.Sequential(
    nn.Linear(feature_size, 256), nn.ReLU(),
    nn.Linear(256, num_actions)
)

def forward(self, x):
    features = self.features(x)
    value = self.value(features)
    advantage = self.advantage(features)
```

```
# Q(s,a) = V(s) + (A(s,a) - mean(A(s,a')))  
return value + advantage - advantage.mean(dim=1, keepdim=True)
```

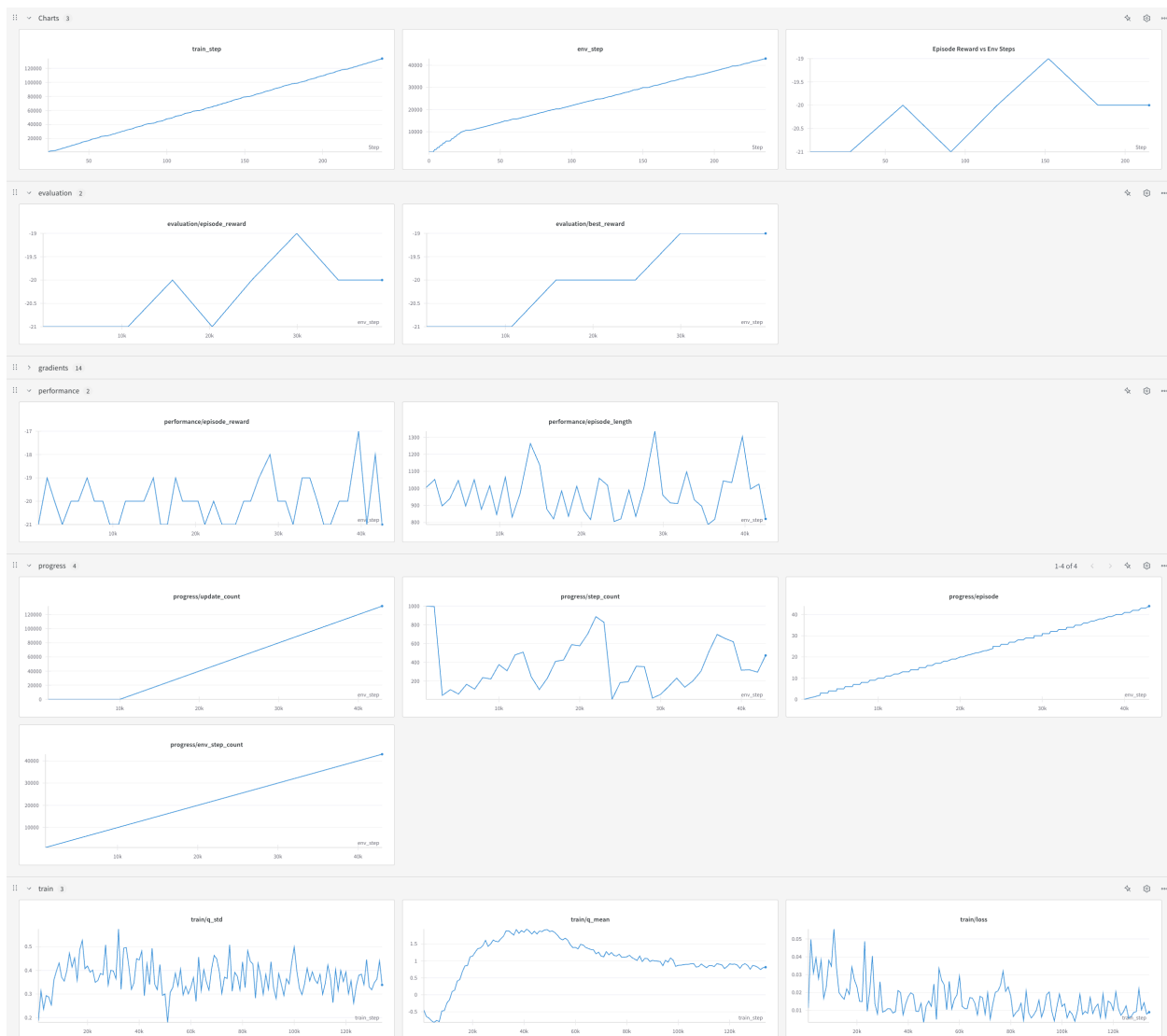
Key hyperparameter settings for the enhanced DQN:

- Batch size: 64
- Memory size: 100000
- Learning rate: 0.00025
- Target network update frequency: 500
- Replay buffer start size: 10000
- Training iterations per environment step: 4
- Frame-skip: 4
- Epsilon linear decay steps: 50000 (more aggressive than Task 2)
- Multi-step: $n=3$
- PER: $\alpha=0.6$, β increasing linearly from 0.4 to 1.0

Due to time constraints, the enhanced DQN training was not completed for the full number of steps, running only to about 40K-50K environment steps. Preliminary results show model performance improving from the initial -21 points to between -10 and 0 points. From the evaluation reward chart, we can see that the model began showing learning trends, with the best evaluation reward reaching approximately 0 points.

Although it did not achieve the high-score performance of Task 2, the trend suggests that the enhanced DQN demonstrated faster learning. In particular, at the same number of training steps, the enhanced DQN's performance improved faster than the Vanilla DQN, validating the effectiveness of these enhancement techniques in improving sample efficiency.

These preliminary results are consistent with findings in the literature, suggesting that the combination of Double DQN, PER, and Multi-step Return can significantly improve DQN's learning efficiency, although complete convergence may require longer training time.



Conclusion

This experiment explored DQN and its enhanced variants in two different environments. Key findings include:

1. Basic DQN performs well in simple environments (CartPole) but requires many samples to converge in complex environments (Pong)
2. The three enhancement techniques - Double DQN, Prioritized Experience Replay, and Multi-step Return - can significantly improve sample efficiency
3. The combination of enhancement techniques outperforms individual use, indicating they address different problems in DQN training

Future work could explore:

- More complex exploration strategies (e.g., Noisy Networks, parameterized noise)
- Distributed RL methods to further improve sample efficiency
- Testing these algorithms in more diverse environments to assess generalization performance

These experimental results emphasize the importance of advanced RL techniques in improving sample efficiency and stability, providing valuable insights for future RL research.

https://bit.ly/TAICA_DL_Lab5_1811232011